

Ingeniería de software

Francisco Javier Quintanilla Moreno



Actividad de aprendizaje 5.2: Manual Técnico

Erick Adan Torres Villarreal

Juan José Castillo García

Manuel Alejandro Ramirez Ramirez

Aimar Alejandro González Placencia

Is-boset Efrén Mejia Mejia

GoogleAI. (2026). GeminiAI (versión Pro 3.1) [Modelo de lenguaje grande].

MANUAL TÉCNICO: THE CHRONICLES OF ARKADIA

Proyecto: RPG Táctico Isométrico

Motor de Desarrollo: Godot Engine 4.x

Lenguaje: GDScript

1. INTRODUCCIÓN

Este manual proporciona las pautas de configuración y la lógica de desarrollo del sistema de información del videojuego *The Chronicles of Arkadia*. Describe la arquitectura modular, el modelo de datos y los algoritmos críticos para asegurar la mantenibilidad y escalabilidad del software.

2. VISTA FUNCIONAL Y LÓGICA (ARQUITECTURA)

El sistema utiliza el patrón arquitectónico Modelo-Vista-Controlador (MVC), adaptado para la estructura de nodos de Godot:

- **Modelo (Capa de Datos):** Gestiona el estado de los personajes, estadísticas de combate y el sistema de inventario. Independiente de la representación visual.
- **Vista (Capa de Presentación):** Controla el renderizado de *pixel art*, las animaciones de los *sprites* y la interfaz de usuario (HUD).
- **Controlador (Lógica de Negocio):** orquesta el flujo de turnos, procesa las entradas del jugador y traduce las acciones en cambios dentro del modelo.

Componentes de Software:

1. **Tactical Engine:** Procesa la cuadrícula isométrica y las reglas de colisión lógica.
2. **Combat Director:** Resuelve los encuentros de combate y el cálculo de daños.
3. **UI Manager:** Gestiona la navegación entre menús y los árboles de diálogo.

3. MODELO DE DATOS

3.1. Modelo Lógico (Entidades)

- **Jugador/Personaje:** Almacena estadísticas (Vida, Energía, Resistencia) y nivel de progresión.
- **SkillCard (Habilidades):** Define el tipo de acción (Ataque/Defensa), valores base y el número de dados (Coins) para el sistema de choque.
- **Misión:** Registra estados de cumplimiento, disparadores (triggers) y recompensas.

3.2. Modelo Físico (Persistencia)

Los datos se almacenan en formato JSON para facilitar el guardado y carga de

partidas.

- Ruta de guardado: `user://saves/`
- Estructura: Diccionarios serializados que mapean el ID de la entidad con sus valores actuales.

4. DESCRIPCIÓN DE ALGORITMOS

4.1. Algoritmo de "Clash" (Choque de Habilidades)

Es el algoritmo principal de combate. Su lógica es la siguiente:

1. Entrada: Se reciben los valores de la carta del atacante y del defensor. **2.** Cálculo de Poder: $\text{Poder} = \text{Valor_Base} + (\text{Lanzamiento_Monedas} * \text{Modificador_Estadística})$.
3. Comparación: El sistema compara ambos valores de Poder.
4. Resolución: El ganador aplica el daño al perdedor. Si hay empate, se activa un subproceso de reintento con penalización de energía.

4.2. Sistema de Navegación A*

Se utiliza el algoritmo A-Star para encontrar el camino más corto en la rejilla isométrica.

- Costo dinámico: El costo de cada casilla aumenta si la "Resistencia" del personaje es baja, limitando su rango de movimiento total.

5. REQUERIMIENTOS TÉCNICOS

5.1. Entorno de Desarrollo

- Godot Engine 4.2.2+ (Versión Standard).
- Git: Para control de versiones y trabajo colaborativo.
- Aseprite: Para la creación y edición de recursos visuales.

5.2. Requisitos de Ejecución (Sistema Mínimo)

- SO: Windows 10/11 o Linux (64 bits).
- Procesador: Dual Core 2.0 GHz.
- Memoria: 4 GB RAM.
- Gráficos: Soporte para OpenGL 3.3 / Vulkan.
- Espacio: 500 MB de almacenamiento disponible.

6. PAUTAS DE CONFIGURACIÓN

Para integrar nuevos elementos al sistema:

1. Personajes: Crear un nuevo recurso `.tres` basado en la clase `CharacterTemplate`.
2. Habilidades: Añadir la entrada correspondiente en el archivo `SkillDatabase.json`.
3. Escenas: Todas las escenas de combate deben heredar de `BattleMap.tscn` para mantener la lógica de nodos preconfigurada.

OpenAI. (2026). ChatGPT (versión GPT-5.2) [Modelo de lenguaje grande].

MANUAL TÉCNICO: THE CHRONICLES OF ARKADIA

Proyecto: RPG Táctico Isométrico

Motor de Desarrollo: Godot Engine 4.x

Lenguaje: GDScript

Arquitectura: Modelo–Vista–Controlador (MVC)

Control de Versiones: Git + GitHub

1. INTRODUCCIÓN

Este manual técnico proporciona las pautas de configuración, estructura lógica y funcionamiento interno del videojuego *The Chronicles of Arkadia*. El documento describe la arquitectura del sistema, el modelo de datos, la construcción de interfaces y los algoritmos principales implementados durante el desarrollo del proyecto.

El objetivo principal es documentar los requerimientos técnicos necesarios para el funcionamiento del sistema, permitiendo mantener una estructura organizada, escalable y mantenible dentro del entorno colaborativo de desarrollo.

2. VISTA FUNCIONAL Y LÓGICA (ARQUITECTURA)

El sistema implementa una arquitectura modular basada en el patrón Modelo–Vista–Controlador (MVC), adaptado al sistema de nodos y escenas de Godot Engine.

2.1 Modelo (Capa de Datos)

Responsable de gestionar toda la lógica y persistencia del sistema:

- Estadísticas de personajes
- Sistema de combate
- Inventario
- Misiones
- Guardado y carga de partidas
- Validación de reglas

La lógica se mantiene desacoplada de la interfaz gráfica para facilitar mantenimiento y pruebas.

2.2 Vista (Capa de Presentación)

Controla todos los elementos visuales y audiovisuales:

- Renderizado pixel art
- HUD del jugador
- Animaciones
- Menús
- Efectos visuales
- Sonido y música

La comunicación con la lógica se realiza mediante señales y eventos del motor Godot.

2.3 Controlador (Lógica de Negocio)

Procesa la interacción del usuario:

- Entradas de teclado y mouse
- Selección de acciones
- Flujo de combate
- Gestión de menús

- **Comunicación entre Vista y Modelo**

3. COMPONENTES DEL SOFTWARE

3.1 Tactical Engine

Subsistema encargado de:

- **Gestión de cuadrícula táctica**
- **Validación de movimientos**
- **Pathfinding**
- **Rango de habilidades**
- **Colisiones**

3.2 Combat Manager

Administra el flujo del combate:

- **Turnos**
- **Selección de objetivos**
- **Cálculo de daño**
- **Aplicación de estados**
- **Resolución de habilidades**

3.3 UI Manager

Gestiona todas las interfaces del sistema:

- **Menú principal**
- **Inventario**
- **Misiones**
- **Party**
- **HUD**
- **Transiciones de interfaz**

3.4 GameManager (Singleton)

Clase global responsable de:

- **Persistencia de datos**
- **Guardado y carga**
- **Gestión de escenas**
- **Estado global del juego**

4. DISEÑO DE INTERFACES

El sistema de interfaces está dividido en tres módulos principales:

- **Menú Principal**
- **Juego (HUD)**
- **Menú del Jugador**

4.1 Menú Principal

Permite iniciar la interacción con el sistema.

Funciones:

- **Nueva partida**
- **Cargar partida**
- **Ajustes**
- **Salir**

Validaciones:

- **Confirmación de sobrescritura**
- **Validación de slots ocupados**
- **Verificación de partidas guardadas**

4.2 HUD (Heads-Up Display)

Interfaz mostrada durante la

partida. Elementos:

- Barra de vida (HP)
- Energía
- Minimapa
- Objetivo actual
- Indicadores de turno

4.3 Menú del Jugador

Accesible durante el combate o exploración.

Inventario

- Consumibles
- Armas
- Armaduras
- Objetos clave

Party

- Gestión de personajes
- Estadísticas
- Equipamiento

Misiones

- Activas
- Finalizadas
- Recompensas

Gestión de Partida

- Guardar
- Cargar
- Salir al menú

5. MODELO DE DATOS 5.1

Modelo Lógico (Entidades)

Jugador

Almacena:

- Nombre
- Nivel general
- Experiencia
- Slots de guardado

Partida

Gestiona:

- Progreso de historia
- Tiempo jugado
- Estado global
- Dificultad

Personaje

Define:

- HP
- Ataque
- Defensa
- Velocidad
- Clase
- Resistencia

Habilidad (SkillCard)

Define:

- Tipo de habilidad
- Daño base
- Costos
- Número de monedas (Coins)
- Efectos especiales

Inventario

Gestiona:

- **Objetos equipados**
- **Cantidades**
- **Capacidad máxima**

Misión

Registra:

- **Objetivos**
- **Estado**
- **Recompensas**
- **Activadores (Triggers)**

5.2 Modelo Físico (Persistencia)

El sistema utiliza persistencia basada en JSON y serialización interna de Godot.

Ruta de guardado:

None

user://saves/

Estructura:

- **Diccionarios serializados**
 - **Datos indexados por ID** •
- Persistencia por slots**

6. MODELO RELACIONAL

Relaciones principales

Relación Cardinalidad

Jugador → Partida 1:N
Habilidad
1:N 1:1

Partida →
Personaje

1:N N:M

Partida →
Inventario

Inventario →
Objeto

Personaje ↔

Partida → Misión 1:N

Entidades principales

Entidad Función Jugador

Gestiona perfiles

Partida Almacena
progreso
tácticas

Personaj e

Gestión de

Habilida d

objetos
Inventari o
Estadísticas y
combate

Acciones

Objeto Ítems utilizables

Misión Objetivos del juego

Enemigo Unidades hostiles

7. DESCRIPCIÓN DE ALGORITMOS

7.1 Algoritmo de Combate

El combate táctico funciona mediante validación de turnos y cálculo de daño.

Flujo:

- 1. El jugador selecciona una acción.**
- 2. El sistema valida rango y posición.**
- 3. Se calcula daño.**
- 4. Se actualizan estadísticas.**
- 5. Se ejecutan animaciones y efectos.**

7.2 Sistema de Clash (Choque)

Entrada:

- Habilidad atacante**
- Habilidad defensora**

Fórmula general:

$\text{Poder} = \text{Valor_Base} + \text{Modificador} * \text{Estadística}$

Resolución:

- Comparación de poder**
- Aplicación de daño**
- Consumo de energía**
- Reintento en caso de empate**

7.3 Sistema de Navegación A*

El Tactical Engine utiliza A-Star para:

- Movimiento**
- Cálculo de rutas**
- Detección de obstáculos**
- Optimización de trayectorias**

Características:

- Costos dinámicos
- Restricción por resistencia
- Validación de colisiones

8. REQUERIMIENTOS TÉCNICOS

8.1 Entorno de Desarrollo

Herramienta Función

4.x

Motor principal

Godot Engine

Git / GitHub Control de

versiones

Code

Edición de

scripts

Visual Studio

PlantUML Diagramas UML

Aseprite Pixel Art

8.2 Requisitos Mínimos

Component e Requerimiento

SO Windows 10/11 o Linux

Procesador Dual Core 2.0 GHz
RAM 4 GB

GPU OpenGL 3.3 / Vulkan

Espacio 500 MB

9. PAUTAS DE CONFIGURACIÓN

Integración de Personajes

Crear un recurso **.tres** basado en:

None

CharacterTemplate

Definir:

- Estadísticas
- Clase
- Sprites
- Habilidades iniciales

Integración de Habilidades

Agregar nuevas habilidades dentro de:

None

SkillDatabase.json

Definir:

- Tipo
- Daño
- Costo
- Animación
- Efectos

Integración de Escenarios

Todas las escenas de combate deben heredar de:

None

BattleMap.tscn

Esto garantiza:

- Compatibilidad con Tactical Engine
- Cuadrícula preconfigurada
- Integración automática con CombatManager

10. SEGURIDAD Y VALIDACIONES

El sistema implementa:

- Validación de movimientos
- Restricción de acciones inválidas
- Confirmación de sobrescritura
- Protección de persistencia
- Control de slots

11. CONCLUSIÓN

La arquitectura modular implementada en *The Chronicles of Arkadia* permite mantener una separación clara entre lógica, presentación y control, facilitando el desarrollo colaborativo y la escalabilidad del sistema.

La integración de MVC, UML y persistencia estructurada proporciona una base sólida para futuras expansiones del videojuego, garantizando mantenibilidad y estabilidad durante el desarrollo.

CONCLUSIONES

Erick Adan Torres Villarreal: Gracias al manual técnico, podemos distribuir de manera visual las funciones del programa, y así, permitir que el usuario tenga acceso a la información técnica del mismo de una manera más clara y comprensible.

Manuel Alejandro Ramírez Ramírez: En esta actividad más que nada se puede y se usa para ayudar al usuario a entender mediante el Manual técnico el cómo funciona el software en términos generales, observar sus funciones, la lógica y tener una descripción detallada sobre el videojuego en sí.

Is-Boset Efren Mejia Mejia: En el proceso de desarrollo aplicado en los videojuegos, por medio del manual técnico se presentan aspectos como la vista funcional y lógica del sistema, los modelos lógico y físico de datos, así como una explicación detallada de los algoritmos y otros elementos importantes del proyecto.

Juan José Castillo García: Se separa los números y las estadísticas de los dibujos como los gráficos, esto permite que el combate sea justo, que los personajes se muevan de forma inteligente por el mapa y que sea muy fácil añadir contenido nuevo sin que el juego se rompa.

Aimar Alejandro Gonzalez Plascencia: Este proyecto demuestra el

desarrollo que se lleva a cabo en videojuegos, integrando de manera efectiva la construcción modular y el aseguramiento de calidad mediante una documentación que se puede llegar a entender. A través del manual técnico podemos observar la vista funcional y lógica, el modelo lógico y físico de datos, descripción detallada de los algoritmos, entre más cosas.

Referencias:

- Pressman, R. S. *Ingeniería de Software: Un Enfoque Práctico*. McGraw-Hill.
- Elmasri, R., & Navathe, S. *Fundamentals of Database Systems*. Pearson.
- Silberschatz, A., Korth, H., & Sudarshan, S. *Database System Concepts*. McGraw-Hill.
- Godot Engine Documentation.